TECHNICAL REFERENCE

GeoQoE Pulse

Dynamic Mobile QoE & Demand Pressure Intelligence System — how the datasets work, how the technology stack fits together, and what every dashboard panel does.

🟢 REAL data & pipeline

🟡 SIMULATED hourly QoE

🤖 AI trained & frozen

Contents

1 How the Datasets Work 3

2 The Technology Stack 5

3 Dashboard Functionality, Panel by Panel 7

4 The AI Training Model, In Full Detail 10

> Every fact in this document was checked directly against the project's source code, README, and generated data files (`run_pipeline.py` , `config.py` , `summary_stats.json` , `model_card.json` , `dashboard.js`) — not written from general knowledge of similar systems.

1 · How the Datasets Work

GeoQoE Pulse combines seven real open geospatial datasets with one simulated hourly layer. Everything is organized around a single spatial index — an H3 hexagon grid — so that a tower location, a building footprint, a road, and an hour of simulated QoE can all be joined together by which hexagon they fall in.

1.1 The spatial backbone: the H3 grid

Before any data is joined, the pipeline draws the map. **Stage 1** loads a study-boundary polygon covering the Klang Valley core — Kuala Lumpur, Putrajaya, and six urbanised Selangor districts (Gombak, Hulu Langat, Klang, Kuala Langat, Petaling, Sepang) — and calls `h3.polygon_to_cells()` to tile it into **4,918 Uber H3 hexagons at resolution 8** (roughly 0.74 km² each). Every dataset that follows is attached to these same 4,918 cells, identified by an `h3_index`. Each cell is also stamped with its resolution-6 parent (about 130 larger groups), which is used later to make sure the AI models are tested on hexagons the model has never seen nearby cells of — not just individually unseen rows.

1.2 The seven real data sources

Stage 2 of the pipeline joins seven independently-sourced open datasets onto the grid, using real spatial operations — `sjoin`, `sjoin_nearest`, `overlay`, and zonal statistics — not approximations.

Source	What it provides	How it's joined
OpenCellID	Cellular base station locations for Malaysia (MCC 502)	Deduplicated to approximate physical sites, then counted per hexagon → `tower_count`, distance to nearest tower
Google Open Buildings V3	Building footprints & built-up area	Overlaid on each hexagon → `built_up_ratio`, building density
OpenStreetMap	Roads, transit stops, points of interest	Parsed via `pyrosm`, joined per hexagon → road density, POI counts (transport, office, retail/food, education, healthcare)
WorldPop Malaysia 2020	Gridded 100m population raster	Zonal statistics (via `rasterstats`) aggregate the raster into per-hexagon day/night population estimates
OpenDOSM Census	Official Malaysian population statistics	Used to sanity-check/calibrate the WorldPop aggregation against official figures
Ookla Open Data (Mobile)	Real mobile download/upload/latency performance tiles	Tiles falling in a hexagon are averaged into a real, static, *quarterly* per-cell measurement — this is the project's only real QoE signal
Public Transit Ridership	Daily LRT/MRT/Monorail ridership	Feeds into demand-pressure context near transit-heavy hexagons

> **Every one of these seven joins is 100% real** — the README states every `*_is_synthetic` flag is `False` across all 4,918 cells. This static layer (`cell_static_features.parquet`, 4,918 cells × 31 columns) contains no QoE data at all — just physical geography.

1.3 The one honest real QoE dataset: the Ookla baseline

Ookla's tiles only cover about **60% of the 4,918 hexagons**, and even where they exist, they're a single quarterly average — not hourly. The pipeline keeps this strictly honest: cells with real tile coverage get real `ookla_dl_mbps` / `ookla_rtt_ms` / `ookla_tests` values; the remaining ~40% are left as genuine `NaN` — **never imputed** — and flagged with `has_real_ookla_label`. This produces `cell_static_qoe_baseline.parquet` (4,918 cells × 35 columns), which the project's own README calls **"the recommended dataset for any modelling claim you want to defend"** — because every feature in it traces back to real GIS data, with zero circularity.

1.4 The simulated hourly panel — and why it exists

No dataset on Earth gave this project real hour-by-hour QoE for 4,918 Klang Valley hexagons over a week — that data simply doesn't exist publicly. So the dashboard's "live" week of data (`cell_time_dataset.parquet`, 826,224 rows = 4,918 cells × 168 hours) is a **documented simulation**: a diurnal-demand formula generates plausible `dl_median` / `ul_median` / `rtt_median` / `jitter_median` / `loss_median` values for every cell, every hour.

It isn't arbitrary, though. Two provenance flags keep the simulation honest and traceable:

- `qoe_level_is_ookla_anchored` — for the ~60% of cells with a real Ookla measurement, the simulated hourly values are **re-centred on that real number**, so the simulated week oscillates around a real baseline rather than a made-up one.
- `qoe_hourly_shape_is_simulated` — always `True`. Even on anchored cells, the specific up-and-down *shape* across the day is generated by the formula, not measured.

> **What this means in practice:** use this dataset for pipeline integration testing, dashboard demos, and building/testing a forecasting *framework* — not for reporting model accuracy as evidence of real-world predictive skill on live network data.

1.5 How "QoE" and "degraded" are actually defined

`target_qoe_index` (0–100) is a **custom composite** — a logarithmic download/upload utility score minus an exponential latency/loss penalty. It is explicitly **not** an ITU-T G.107 E-model MOS, and not a P.1203/P.1204 implementation. Degradation is *not* derived from a cutoff on this index — it comes independently from raw thresholds on `dl_median` / `rtt_median`:

- **Degraded** (class 1) if download < 10 Mbps **or** RTT > 100 ms
- **Severe** (class 2) if download < 5 Mbps **or** RTT > 150 ms

Both the QoE score and the degradation class come from the same threshold function in one call, so — by construction — they can never contradict each other, which is exactly why a cell can show a mediocre 53-point QoE score while still being correctly classified "Normal": the composite score and the raw-metric threshold check are deliberately independent measurements of the same cell.

1.6 Derived datasets: demand pressure, hotspots, and the AI layer

Stage	Output	What it computes
3	Temporal features	Leakage-free lags/rolling stats — `shift()` only, never back-filled, with `*_valid` flags marking real warm-up gaps
4	Demand Pressure Index	`demand_pressure_base` (real, static-feature-driven) plus a historical-QoE component explicitly flagged as circular with the simulated panel
5	Spatial features	Real 1-ring adjacency via `libpysal` — spatial lags, variance, rolling mean of neighbouring cells
6	Hotspot detection	Real `esda.G_Local` (Getis-Ord Gi*) and `esda.Moran_Local`, 99 permutations, genuine p-values, HH/LH/LL/HL quadrant labels
7	QoE targets & forecasting	The composite QoE index, degradation classes, and a genuine 1-hour-ahead forecast target via `groupby.shift(-1)`
8	Validation	Structural + statistical regression guards — hard errors if the two degradation labels ever disagree

On top of this, two XGBoost models (covered in full in Section 2.4) are trained and frozen at a single snapshot hour, producing the "AI Model" layer the dashboard displays separately from everything else.

1.7 Known limitations (stated by the project itself)

- No real per-hour QoE data exists anywhere in the project — the hourly panel is a labelled simulation.
- No real network capacity/traffic data exists — demand-pressure features are activity proxies, not confirmed congestion measurements.
- `tower_count` is an approximate deduplication of OpenCellID records, not an authoritative operator inventory.
- The QoE index is a custom composite, not a validated ITU-T G.107 MOS.
- `demand_pressure_index` (not `_base`) is circular with the simulated panel via its historical-RTT component.
- On the simulated panel, positive "will degrade" labels concentrate in Ookla-anchored cells — `qoe_level_is_ookla_anchored` is a near-trivial predictor and is hard-excluded from the model's feature list.

2 · The Technology Stack

Six layers, each verified directly against the source code — not just `requirements.txt`. Two commonly-listed tools turned out not to be used anywhere in the code, and one real tool was missing from the original list.

Layer	What's actually used (verified)
Data Processing	Python, **GeoPandas** (heavily — all joins, geometry, CRS reprojection), **pyrosm** (parses the OpenStreetMap `.pbf` file), **rasterstats** (zonal statistics on the WorldPop raster)
Spatial Grid	Uber **H3**, resolution 8
Spatial Statistics	**libpysal** + **esda** (Local Moran's I, Getis-Ord Gi*)
Machine Learning	**XGBoost**, **scikit-learn** (metrics, preprocessing, baselines)
Dashboard	**Leaflet.js**, **Plotly**, **Bootstrap**, plain HTML/CSS/JS
Server	Python `http.server` or Node.js (`server.js`) — both confirmed present, either works

2.1 Two corrections worth knowing about

Rasterio & PyProj — used indirectly, not directly. The code never calls either library itself. Population raster reading goes through `rasterstats.zonal_stats()` (which depends on Rasterio internally), and all CRS reprojection goes through GeoPandas's `.to_crs()` (which depends on PyProj internally). If you're listing tech stack for a pitch, it's more accurate to name **GeoPandas** and **rasterstats** as the direct tools, with Rasterio/PyProj as dependencies underneath them.

SHAP — not used anywhere. A repo-wide search for `import shap` or `shap.*` calls turns up nothing; the only text matches were false positives from `shapely` (the geometry library) and `X_train.shape`. If model explainability is on the roadmap, SHAP would need to be added — it isn't in `requirements.txt` for either the pipeline or the model-training package.

One real tool that was missing from the original list: `pyrosm` — this is what actually parses the OpenStreetMap `.pbf` extract into roads, transit stops, and POIs in Stage 2 of the pipeline.

2.2 Data Processing layer, in more detail

GeoPandas is the workhorse of the whole pipeline: every spatial join (`sjoin`, `sjoin_nearest`, `overlay`), every geometry operation, and every CRS transform runs through it. The project enforces a strict CRS discipline — data is stored/exported in WGS84 (`EPSG:4326`) but every distance, area, and length calculation is performed after reprojecting to Peninsular Malaysia RSO (`EPSG:3375`), never in raw degrees. **pyrosm** handles the one format GeoPandas can't read natively — the compressed OpenStreetMap `.pbf` extract. **rasterstats** bridges the gap between the WorldPop population *raster* (a grid of pixels) and the H3 *vector* grid (a set of hexagon polygons), aggregating pixel values inside each hexagon into a single population estimate.

2.3 Spatial Grid & Spatial Statistics layers

H3 (resolution 8) is the indexing system everything else keys off — a hierarchical hexagonal grid where every cell has a well-defined set of neighbours, which is exactly what the next layer needs. **libpysal** builds a real 1-ring spatial adjacency graph on top of the H3 grid (which cells touch which), and **esda** runs two classic spatial statistics on that graph every hour: **Getis-Ord Gi*** (is this cell part of a statistically significant cluster of similarly bad/good neighbours?) and **Local Moran's I** (does this cell agree or disagree with its neighbours, producing the HH/LL/HL/LH quadrant classification). Both use 99 real permutations to generate genuine p-values — not approximated or hand-waved significance.

2.4 Machine Learning layer

Two independent **XGBoost** models are trained (with **scikit-learn** used for splitting, scaling, and evaluation metrics around them), both frozen at a single snapshot hour (2026-06-30 23:00 UTC) rather than run live:

Model	Job	Validated performance
Static QoE Baseline	Estimates download speed (Mbps) for the ~40% of cells with no real Ookla label	R^2 = 0.021 (near-random — openly disclosed), MAE = 82.3 Mbps

Next-Hour Onset Model	Ranks which currently-healthy cells are about to degrade in the next hour	ROC-AUC = 0.996 (excellent ranking), Precision = 12.5%, Recall = 99% (final held-out test: 103 true positives, 718 false positives, 1 false negative, 61,297 true negatives)

Both models are deliberately used as **ranking tools**, not binary alarms — the dashboard's AI Forecast panel explicitly labels its output "NOT a %" and presents an alert-volume trade-off (Top-20/50/100-per-hour) rather than a single yes/no threshold, precisely because precision is low even though recall is excellent.

2.5 Dashboard layer

Leaflet.js renders the interactive hexagon map and all its color-coded display modes. **Plotly** renders every chart — the time-series panels, the root-cause bar chart, the AI-vs-traditional comparison. **Bootstrap** provides base layout/component styling, with a substantial custom `dashboard.css` on top for the specific card/panel design. Everything is plain **HTML/CSS/JS** — no frontend framework (no React/Vue) — which is also why the whole site can be served as static files with no build step at runtime.

2.6 Server layer

Because every number the dashboard shows is pre-computed at build time into static JSON/GeoJSON files, the "server" has almost nothing to do — it just needs to hand out static files. The repo supports two interchangeable options: Python's built-in `http.server` module (`python3 -m http.server 3000`, zero dependencies) or a small Node.js script (`server.js`). Neither runs any model inference, database query, or per-request computation — **there is no live API call anywhere in this system**, which is a deliberate architectural choice, not a missing feature.

3 · Dashboard Functionality, Panel by Panel

GeoQoE Pulse is a mobile network monitoring dashboard for the Klang Valley. It shows, hexagon by hexagon, how good mobile internet quality is, where it's degraded, why, and what to do about it — across a simulated week of hourly data plus the trained AI model layer described above.

3.1 The honesty-badge system

Three color tags appear throughout the dashboard, and they are the single most important thing to understand about how to read it:

- REAL — actual measured data: cell towers, buildings, roads, population, real Ookla speed tests.
- SIMULATED — the hour-by-hour QoE numbers are a documented, realistic simulation, not live telemetry.
- AI MODEL — output from an actually-trained model, scored once for a single frozen hour, not re-run as the time slider moves.

3.2 Header

Region badge, time slider, export, dark mode

The header shows the fixed study region ("Klang Valley core"), a ▶ Play button and time slider that scrubs through all 168 simulated hours — every chart, map color, and KPI updates live to whichever hour is selected — an "Export Executive Summary" button that opens a print-ready one-page PDF-able report, a dark-mode toggle, and a timestamp showing when the underlying scenario data ends.

3.3 KPI cards

Top row — the 30-second summary

Network Health (overall QoE score with a trend arrow), Degraded Cells (count + % of all 4,918), the AI Forecast's latest read, the single worst-performing area right now, the most common root cause across degraded cells, and a one-line recommended action.

Second row — supporting metrics

Peak Demand Pressure, Population Affected, Total Cells, and % of cells anchored to a real Ookla measurement — deliberately non-overlapping with the top row so nothing is shown twice.

3.4 Live QoE Map — the Display dropdown

The core of the dashboard. The Display dropdown changes what the hexagon colors mean — same map, same cells, different lens:

Mode	What it shows
QoE Score (0–100)	Overall quality, red→green. The default view.
Demand Pressure (0–100)	How loaded a cell is relative to capacity — different from QoE; a cell can be busy but fine, or quiet but still degraded.
Hotspot Classification (Gi*)	Is this cell part of a statistically significant cluster of bad/good neighbours, from `cold_99` through `not_significant` to `hot_99`.
Local Moran's I Cluster	HH (bad cell, bad neighbours — a real problem cluster), LL (healthy cluster), HL (isolated one-off bad cell, good candidate for a quick fix), LH (isolated good cell in a bad area — often turns HH next).
Next-Hour Risk (historical)	Look-back ground truth — did this cell actually degrade the following hour, historically. Not a live prediction.
🤖 AI Onset Risk	The trained model's real prediction of what's about to degrade, frozen at the one snapshot hour.
🤖 AI Static DL Estimate	Real Ookla speed where measured, AI-estimated speed filling the gaps elsewhere.
🔍 AI vs Traditional	Comparison mode — red = both methods flag it, orange = AI-only, yellow = traditional-only, gray = neither.

Modes 1–5 recompute live for whatever hour the time slider is on. Modes 6–8 are all tied to the AI model's one frozen snapshot hour regardless of the slider position.

3.5 Live QoE Map — other controls

Profile dropdown

Reweights the QoE formula for a use case — e.g. a video-calling profile weights latency more heavily than a bulk-download profile would.

Map style

Switches the basemap, and toggles cell tower markers and a live rain-radar overlay (real current weather, not tied to the simulated panel).

Compare cells

Click to enter comparison mode, then click hexagons on the map to build a side-by-side table/chart in the right column.

Search bar

Find a cell by real place name (e.g. "Cyberjaya") or its technical H3 index, and jump the map straight to it.

🪄 What-if box

Simulates a network capacity change (−50% to +100%) using the pipeline's real demand-elasticity math, and shows live in the map/KPIs how many fewer cells would be degraded — tagged "model-based estimate," not a live network test.

Confidence filter

Some hexagons have very few real samples behind their reading, so a single noisy measurement can look like false degradation. This slider hides low-confidence cells from the ranked *lists* only — the map itself always shows every cell, so spatial coverage is never lost, only the action lists get cleaner.

3.6 🔥 Spatial-Temporal Hotspots

Four ranked, clickable lists:

- **Worst QoE right now** — the lowest-scoring cells this hour.
- **Highest demand pressure** — the busiest cells this hour.
- **Persistent hotspot watch** — cells that were a statistically significant Gi* hotspot in at least 3 of the last 6 hours, shown as "N/6h hot." This flags chronic, structural problems rather than one-off spikes.
- **Emerging** — cells whose QoE dropped more than 20% over the last 2 hours but aren't yet a confirmed hotspot — an early-warning list.

3.7 🧭 Root Cause Breakdown

A client-side heuristic — explicitly labelled as one, not a model output — that estimates why each currently-degraded cell is bad, using five real physical signals: demand pressure (congestion), distance to nearest tower (coverage gap), built-up ratio (indoor attenuation), a latency-minus-congestion residual (backhaul limitation), and how much confidence exists in the reading at all (insufficient data). Because it's rule-based on real features rather than a black-box prediction, every flag can be explained in one sentence — "coverage gap because the nearest tower is over 1km away and the area is heavily built up."

3.8 🧪 Data Quality & Trust

Spells out, in the dashboard itself, exactly which datasets are real (all seven static sources), what fraction of the hourly panel is Ookla-anchored (60%), and states plainly that the remaining 40% is simulated-only. This panel exists specifically so a viewer never has to guess what's measured versus modelled.

3.9 ⚖️ Cell Comparison

Appears once cells are picked via "Compare cells" on the map — a side-by-side table plus a 7-day trend line chart, useful for distinguishing a temporary blip (a line that dips for one hour) from a structural problem (a line that stays low every single day).

3.10 Time-Series Analytics

A dual-handle range slider zooms into any part of the 168-hour week; an Hourly/Daily toggle smooths the view. Three charts: average QoE over time (dips during the day, recovers at night), average demand pressure (mirrors QoE inversely — the mathematical proof of the demand-capacity relationship), and a stacked Normal/Degraded/Severe bar chart showing the human cost of the worst hours. Clicking any point on any chart jumps the map to that hour.

3.11 🤖 AI Forecast — Latest Available Model Run

A banner makes clear the model has only ever been scored once, for one specific UTC hour — it is not live. The panel is deliberately upfront about the model's real weaknesses (12.5% precision, meaning roughly 7 false alarms for every correct one) alongside its real strength (99% recall — it almost never misses a genuine failure). Because of that trade-off, the output is explicitly presented as a *ranking*, not a probability — the UI literally states "NOT a %" next to the risk score, and offers an alert-budget table (Top-20/50/100-per-hour) so an operator can choose their own precision/recall trade-off rather than being handed a false sense of certainty.

3.12 🔍 AI vs Traditional Hotspot Detection

Directly compares the AI model's flagged cells against the classic Gi* statistical method at the AI's one snapshot hour, broken into four counts: both agree, AI-only, traditional-only, neither. The AI-only cells are the interesting ones — places that look fine on a standard heatmap right now but are trending toward failure; the traditional-only cells are the "stable but chronically bad" cells the AI correctly learns not to escalate.

3.13 🛠️ Intervention Priorities

The dashboard's final output — a ranked, full-width to-do list of currently degraded cells, ordered by the pipeline's real `priority_base_score` (severity combined with population affected and how long the problem has persisted). Each card shows the evidence (QoE, demand, RTT, persistence, AI risk status) and a recommended action with an illustrative cost range (e.g. survey a low-confidence cell for a few thousand Ringgit before recommending a few-hundred-thousand-Ringgit tower build). Engineers can mark cells planned/investigating/resolved, saved to the browser's local storage — a lightweight built-in project tracker with no separate spreadsheet required.

> **The throughline across every panel:** nothing on this dashboard hides what kind of number it's showing you. A QoE score, a root-cause bar, and an AI risk rank are all clearly and separately labelled as real, simulated, or model-derived — which is what makes the Intervention Priorities list at the end trustworthy enough to actually act on.

4 · The AI Training Model, In Full Detail

Two independent XGBoost models, trained through two genuinely different pipelines with different cross-validation strategies — because they're solving two different kinds of problem. This section documents exactly how each was built, split, weighted, and evaluated, pulled directly from the training scripts (`static_train_xgboost.py`, `onset_train_xgboost.py`, `onset_feature_registry.py`, `temporal_split.py`) and the frozen model card.

4.1 Model 1 — Static QoE Baseline (spatial regression)

Job: estimate real-world Ookla download speed (Mbps) for the ~40% of hexagons that have no real Ookla measurement, so the map never has a blank hole where a real number just doesn't exist.

Training data & target

Trained only on `cell_static_qoe_baseline.parquet` — the ~60% of the 4,918 cells that carry a real `ookla_dl_mbps` label. The target is that real Mbps figure directly; nothing simulated ever enters this model's training data.

Features — 16 real static GIS variables

Group	Features
Infrastructure (2)	`tower_density_per_km2`, `dist_to_nearest_tower_m`
Buildings (3)	`building_density_per_km2`, `built_up_ratio`, `avg_building_footprint_m2`
Points of interest (6)	education, healthcare, retail/food, office, recreation, transport-stop counts
Accessibility (3)	`road_density_km_per_km2`, `intersection_density_per_km2`, `dist_to_nearest_transit_m`
Population activity (2)	`population_density_per_km2`, `population_day`

Several near-duplicate columns were deliberately excluded rather than left in — e.g. `opencellid_record_count` was dropped in favour of `tower_density_per_km2` because the two are almost perfectly correlated, and feeding a model two versions of the same signal just dilutes feature importance without adding information.

Confidence weighting — the "confidence-weighted" in the model's name

Not every real Ookla label is equally trustworthy — a cell averaged from 200 real speed tests is a more reliable target than one averaged from 3. So each training row is weighted by its own `ookla_tests` count (the `SAMPLE_WEIGHT_COLUMN`) — cells backed by more real measurements pull the model harder during training than cells backed by only a handful.

Validation — 5-fold spatial cross-validation, not random

Uses scikit-learn's `GroupKFold` with 5 folds, grouped by `h3_parent_res6` (the same ~130 larger H3 regions used throughout the project) rather than a random row split. This matters: mobile QoE is spatially autocorrelated — a naive random split would let the model "cheat" by training on one hexagon and validating on its next-door neighbour, which shares almost identical infrastructure. Holding out entire regions instead forces the model to generalize to genuinely unseen areas.

The five fold models are then combined into an **ensemble**: the final estimate for every cell (`ai_ensemble_estimate_mbps`) is the mean of all 5 folds' predictions, and the spread between them (`ensemble_std_mbps`) becomes a disagreement signal — high spread across the 5 folds means the model isn't confident, which the dashboard surfaces as `ensemble_disagreement_level` (low/moderate/high) rather than a fabricated accuracy percentage.

Validated performance

Metric	Value	What it means
Spatial out-of-fold MAE	≈ 82.3 Mbps	On average, an estimate is off by about 82 Mbps in either direction
R^2	0.021	Essentially no better than predicting the average for every cell — openly disclosed as a rough estimate only, not a validated predictor

> The project's own documentation is blunt about this: *"Ookla-labelled and unlabelled cells have strong GIS coverage-selection differences"* — meaning the ~40% of cells without a real label aren't a random sample of the ~60% that do have one, which is a structural reason this model's accuracy is limited no matter how it's tuned.

4.2 Model 2 — Next-Hour Onset Model (classification / ranking)

Job: look at a hexagon that is currently healthy and rank how likely it is to degrade in the next simulated hour.

Three model variants were actually trained and compared

This wasn't a single model — three different feature sets were trained and evaluated side by side, each testing a different hypothesis about what predicts an onset:

Variant	Features (41 total pool)	Hypothesis being tested
`time_only`	5 — hour of day, day of week, weekend flag, peak-period flag, ridership surge factor	Can time-of-day alone predict onsets? (a naive baseline)
`qoe_history_only`	15 — 5 current QoE readings (dl/ul/rtt/jitter/loss) + 10 lagged/rolling QoE stats (1h/2h/24h lags, 24h rolling mean & std of dl and rtt)	Does a cell's own recent trajectory predict what happens next?
`full_causal` (selected)	41 — everything above **plus** all 16 static GIS features, `demand_pressure_base`, and 4 same-hour spatial-lag/variance features from neighbouring cells	Does adding geography and neighbourhood context on top of history improve things?

Both XGBoost and logistic-regression versions of each variant were trained, giving 5 models to compare on equal footing before anything was frozen.

Validation — chronological split, not random

A random row split would let the model train on 2pm Tuesday and validate on 3pm the same Tuesday — leaking the answer, since adjacent hours are highly correlated. Instead, `temporal_split.py` sorts all distinct forecast timestamps chronologically and cuts them **70% train / 15% validation / 15% test** — so the model is always validated and tested on time periods strictly later than anything it trained on, the same discipline a real production forecasting system would need.

Class weighting — why recall is so high and precision so low

Onset events are rare and clustered almost entirely around the 07:00 and 16:00 peak-demand hours. Left alone, a model would learn to just always predict "no onset" and still be right the vast majority of the time — technically accurate, operationally useless. To counter this, XGBoost's `scale_pos_weight` is set to the ratio of negative to positive training examples, which tells the model to treat every rare positive case as proportionally more important. This is exactly why the frozen model catches 99% of real onsets but also fires many false alarms — the weighting deliberately trades precision for recall, and as a direct side effect, the raw output score is **no longer a calibrated probability** (a documented, intentional trade-off, not an oversight).

Model selection — chosen on validation, before ever touching the test set

The `full_causal_xgboost` variant was selected because it had the **highest validation PR-AUC** — decided and locked in before the held-out final test set was touched at all.

> **A methodological detail worth calling out on its own:** the project's own README states that `qoe_history_only` actually scored a *higher* PR-AUC on the final test set than the selected `full_causal` model — but the team kept `full_causal` frozen anyway, because it had already won on validation before the test set was ever opened. Switching to whichever model happened to score best on the test set after the fact would be a subtle form of test-set leakage — the model card documents this discrepancy openly instead of quietly picking the better-looking number.

Frozen model specification

Selected model	`full_causal_xgboost`
Features	41
Frozen best iteration	18
Frozen validation threshold	0.4477 (selected on validation PR-AUC, applied unchanged at test time)
Snapshot scored	2026-06-30 23:00:00 UTC — 2,843 eligible cells (cells not anchored to a real Ookla label are out of scope and marked `not_scored`)

Final held-out test performance

Metric	Value
ROC-AUC	0.9955
PR-AUC	0.1610

Precision	12.5%
Recall	99.0%
F1	0.223

	Actually degraded	Actually fine
Predicted high-risk	103 (true positives)	718 (false positives)
Predicted OK	1 (false negative)	61,297 (true negatives)

The operational fix: rank, don't alarm

Because a binary alarm at the frozen threshold means roughly 7 false alerts for every real one, the dashboard never presents this as a yes/no alarm. It's exposed as an **alert-budget trade-off** instead — an operator picks how many cells per hour they can actually investigate, and the model's ranking (not its raw score) decides who's on that list:

Alert budget	Real onsets caught	Total alerts generated
Top 20 / hour	46 of 104 (44%)	~440 / hour across the test window
Top 50 / hour	103 of 104 (99%)	~1,100 / hour
Top 100 / hour	104 of 104 (100%)	~2,200 / hour

This is also why the risk score is explicitly labelled **"not a %"** everywhere it appears on the dashboard — a score of 0.64 means "ranked ahead of roughly 64% of other eligible cells," never "64% chance of failure."

Operational status tiers

Rather than exposing the raw score, the dashboard maps every eligible cell into one of six tiers, checked in this order:

1. `current_degradation` — already degraded right now
2. `highest_priority` — top relative-risk band
3. `high_priority` — next risk band down
4. `elevated_watch` — above the frozen threshold, but outside the top bands
5. `routine_watch` — eligible and scored, but low relative rank
6. `not_scored` — outside the onset model's scope entirely (not Ookla-anchored)

Documented limitations (from the model card, stated plainly)

- Onsets are extremely rare and heavily concentrated at the 07:00 and 16:00 peak hours — the model has comparatively little to learn from for other hours.
- Class weighting makes the score explicitly uncalibrated — it must never be shown as a percentage.
- The frozen model is dominated by current and lagged QoE features — geography and demand context help, but recent history is still doing most of the work.
- Both models require real hourly telecom telemetry, longer time coverage, and external validation before any operational use beyond this prototype/demo.